seaborn & plotnine for Python

Workshop 2 — Module 7

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. Three Python viz libraries: matplotlib, seaborn, plotnine
2. seaborn: figure-level vs axes-level functions
3. seaborn gallery: scatter, box, violin, histogram, heatmap, pairplot
4. plotnine: the ggplot2 clone for Python
5. Decision tree: which library for which task
6. R $\leftrightarrow$ Python Rosetta Stone

Learning Outcome
You will choose the right Python viz library for each task and translate any ggplot2 chart into seaborn or plotnine with minimal code changes.

Three Python Libraries

matplotlib vs seaborn vs plotnine

Three Python Visualization Libraries: When to Use Each

matplotlib

Low-level
Figure/Axes/Artists
Full manual control
Verbose but flexible

```
OO API:
fig, ax = plt.subplots()
ax.scatter(x, y)
```

seaborn

Statistical viz
Built on matplotlib
Auto-aggregation
FacetGrid, themes

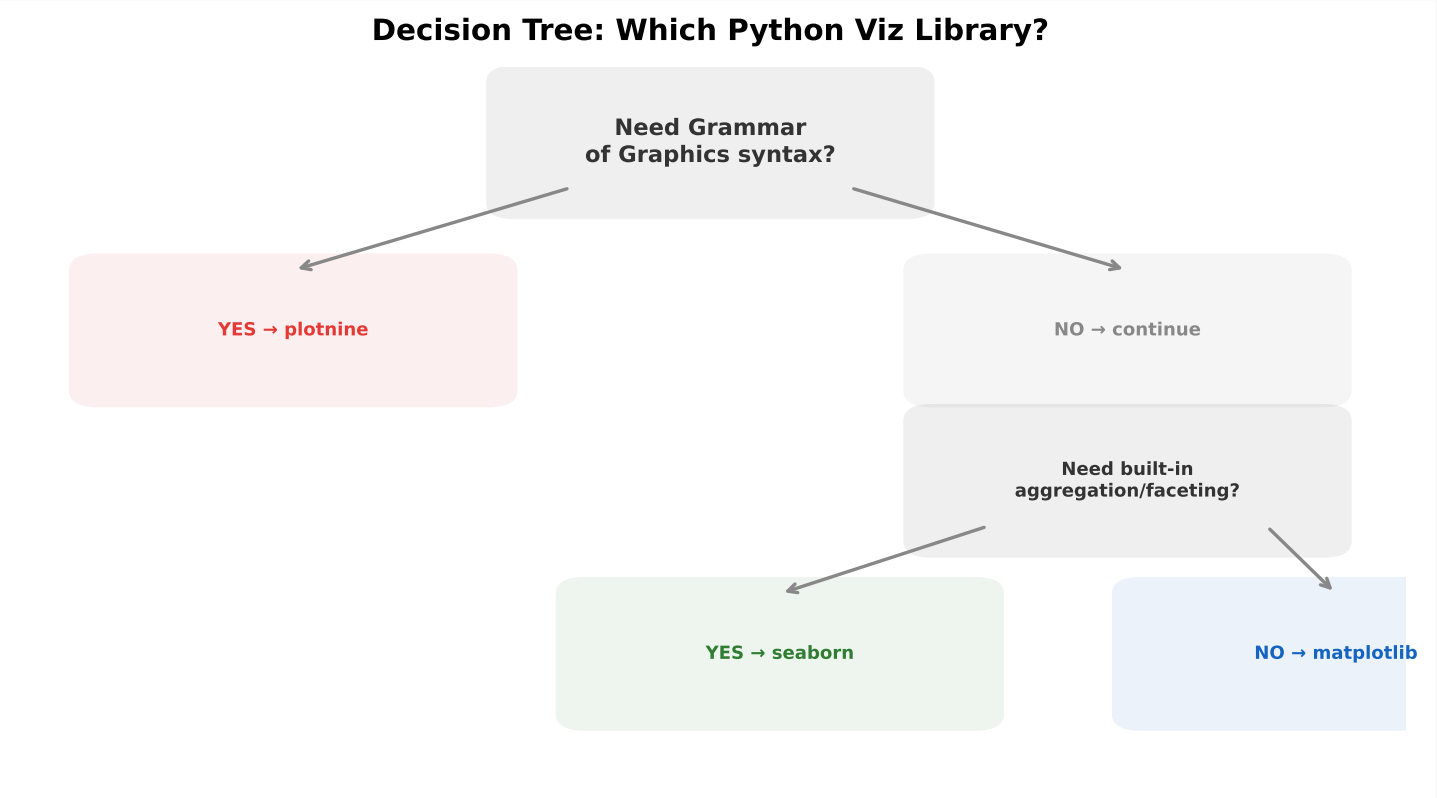
```
Function API:
sns.scatterplot(data=df,
x='col1', y='col2',
hue='col3')
```

plotnine

ggplot2 clone
Grammar of Graphics
Identical syntax to R
+ operator layering

```
GoG API:
(ggplot(df, aes('x','y'))
+ geom_point()
+ theme_minimal())
```

When to Use Each



Pro-Tip

If you are already fluent in ggplot2 (R), start with **plotnine** — the syntax is identical. If you want built-in statistical aggregation and faceting, use **seaborn**. Fall back to **matplotlib** only for full manual control or highly customised layouts.

seaborn

Function Taxonomy: Figure-Level vs Axes-Level

seaborn Function Taxonomy: Figure-Level vs Axes-Level

Level	Function	Chart Types	Key Features
Figure-level	`sns.relplot()`	Scatter, line	kind='scatter'/'line' Faceting built-in: col=, row=
Figure-level	`sns.catplot()`	Strip, box, violin, bar, count, point	kind='box'/'violin'/... Faceting built-in
Figure-level	`sns.displot()`	Hist, KDE, ECDF	kind='hist'/'kde'/'ecdf' Faceting built-in
Axes-level	`sns.scatterplot()`	Scatter on existing ax	ax=ax parameter More control, no faceting
Axes-level	`sns.boxplot()`	Box on existing ax	ax=ax parameter
Axes-level	`sns.heatmap()`	Heatmap	annot=True, cmap='viridis'

Figure-Level: Built-In Faceting

```
import seaborn as sns

# Figure-level: creates its own Figure + facets automatically
g = sns.relplot(
    data=apps, x="Reviews", y="Rating",
    hue="Type",                    # colour aesthetic
    col="Content_Rating",          # facet by column
    col_wrap=3,                    # wrap after 3 columns
    kind="scatter",                # scatter (or "line")
    height=3, aspect=1.2,          # panel size
    alpha=0.3, s=15)
g.set(xscale="log")               # log scale on all panels
g.set_titles("{col_name}")        # strip label format
g.tight_layout()
g.savefig("faceted_scatter.png", dpi=300)
```

Data Insight

Figure-level functions (`relplot`, `catplot`, `displot`) create the entire figure and handle faceting with `col=` and `row=`. Axes-level functions (`scatterplot`, `boxplot`) plot onto a single existing axes — use them inside `plt.subplots()`.

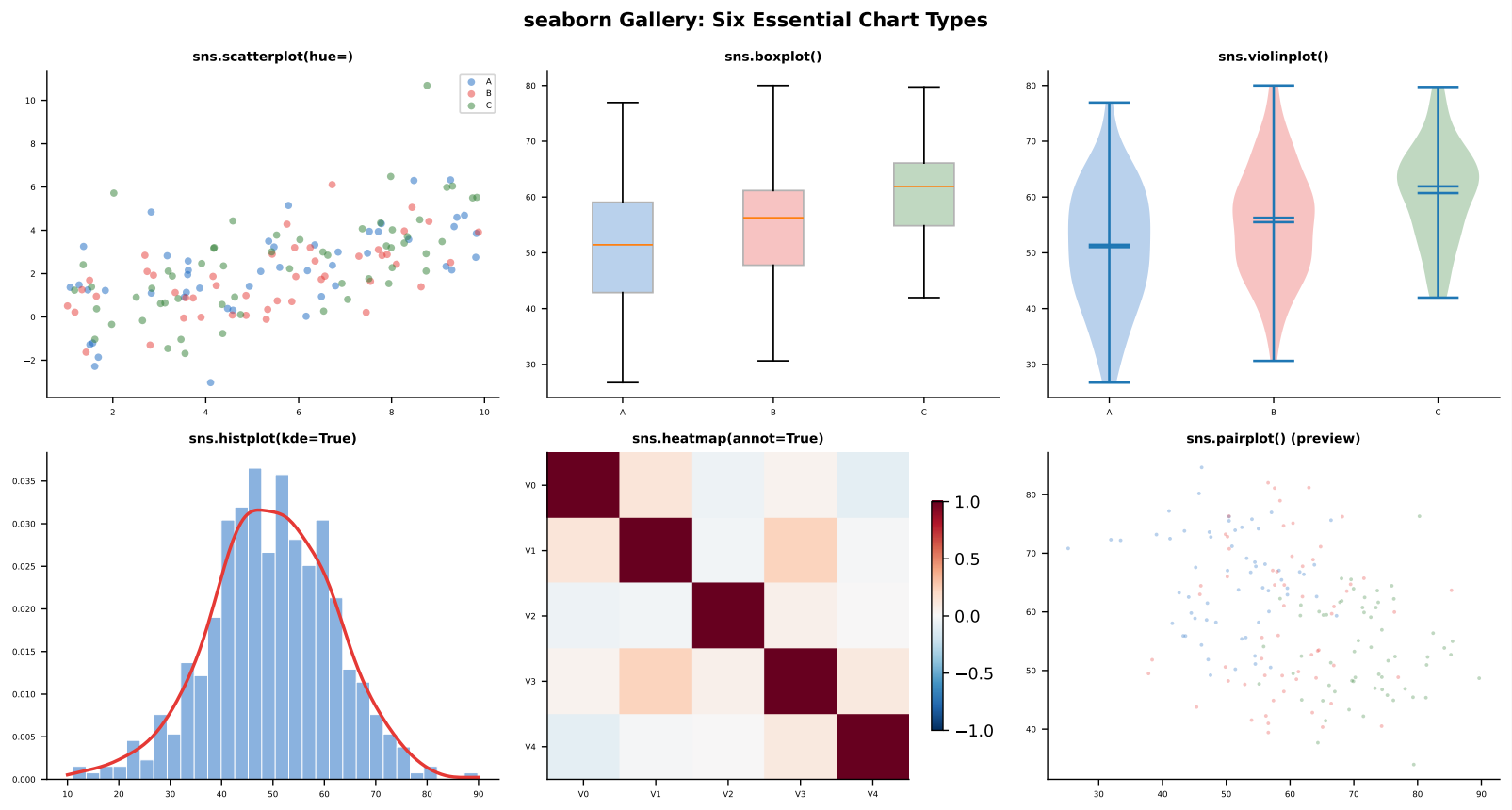
Axes-Level: Embed in Custom Layouts

```
# Axes-level: plot onto an existing axes
fig, axes = plt.subplots(1, 3, figsize=(12, 4), sharey=True)

# Panel 1: scatter
sns.scatterplot(data=df, x="x", y="y", hue="group", ax=axes[0])
axes[0].set_title("Scatter")

# Panel 2: box
sns.boxplot(data=df, x="group", y="y", palette="Set2", ax=axes[1])
axes[1].set_title("Box")

# Panel 3: violin
sns.violinplot(data=df, x="group", y="y", palette="Set2", ax=axes
    [2])
axes[2].set_title("Violin")

plt.tight_layout()
```

Pro-Tip

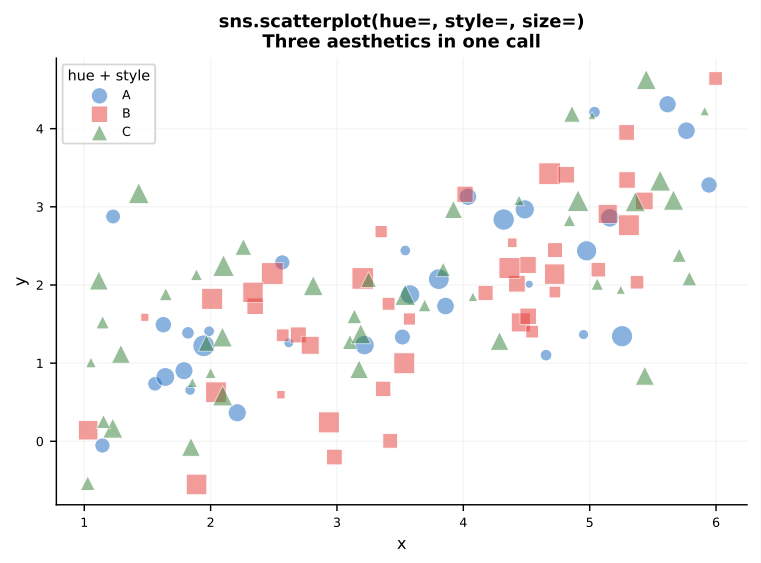
The ax= parameter is the key: it tells seaborn to draw on your axes instead of creating a new figure. This is how you combine seaborn with matplotlib's `GridSpec` for custom layouts.

seaborn Gallery: Six Chart Types



seaborn: Multi-Aesthetic Encoding

```
sns.scatterplot(
    data=df,
    x="gdp",
    y="life_exp",
    hue="continent",      # colour
    style="income",       # shape
    size="population",    # size
    sizes=(20, 300),      # size
        range
    alpha=0.6,
    palette="Set2")
```

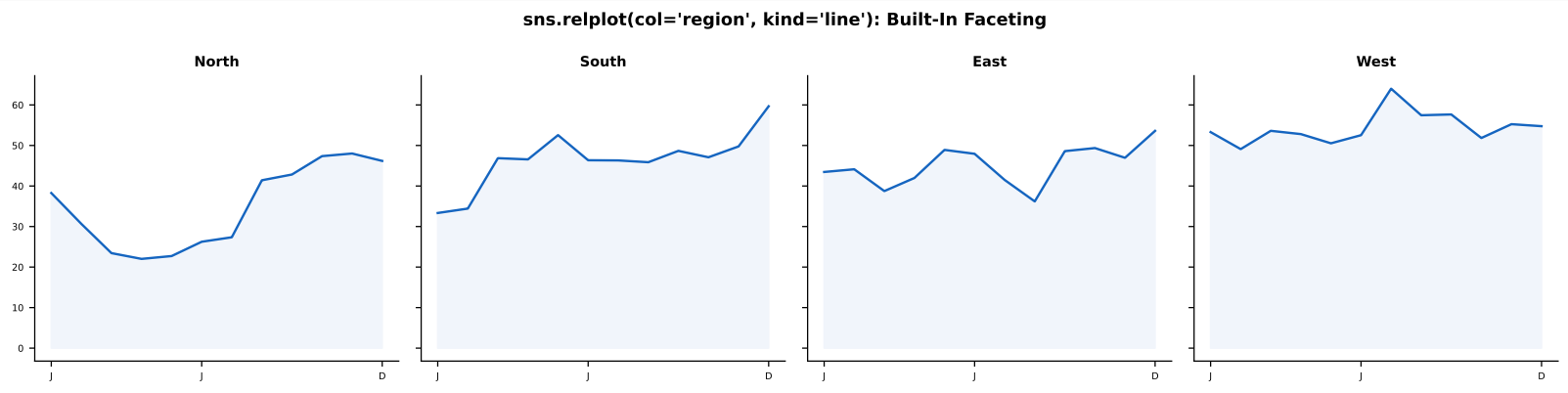


seaborn's `hue`, `style`, and `size` map directly to ggplot2's `aes(color=, shape=, size=)`. Legends are generated automatically.

seaborn: Statistical Aggregation

```
# catplot with built-in mean + CI (like stat_summary in ggplot2)
g = sns.catplot(
    data=apps, x="Category", y="Rating",
    kind="bar",          # computes mean + CI automatically
    ci=95,               # 95% confidence interval
    palette="Set2",
    height=5, aspect=1.5)
g.set_xticklabels(rotation=45, ha="right")

# regplot: scatter + regression line + CI (like geom_smooth)
sns.regplot(data=df, x="x", y="y",
    ci=95, scatter_kws={"alpha": 0.3, "s": 15},
    line_kws={"color": "#E53935"})

# lmplot: regplot + faceting
sns.lmplot(data=df, x="x", y="y",
    hue="group", col="region", col_wrap=3,
    ci=95, height=3, aspect=1.2)
```

seaborn FacetGrid: Built-In Faceting



plotnine

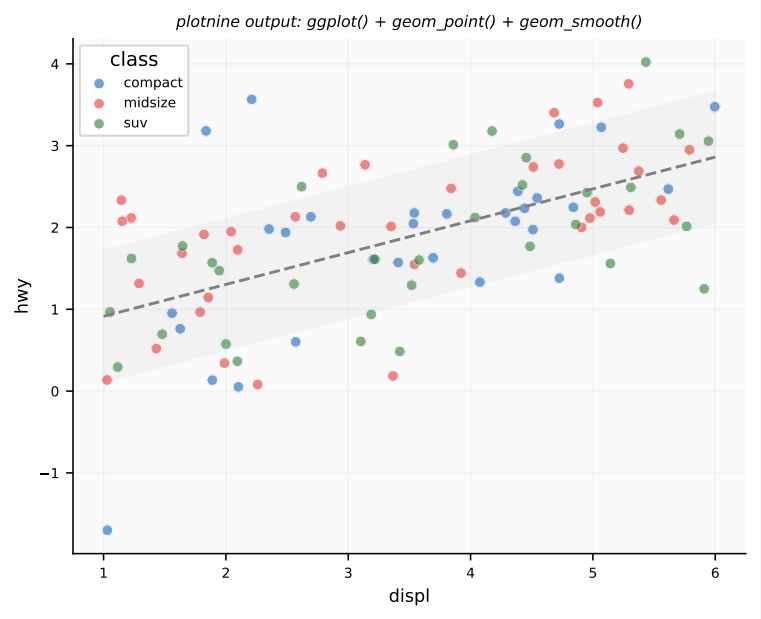
ggplot2 → plotnine: Line-by-Line

ggplot2 (R) vs plotnine (Python): Line-by-Line Comparison

Layer	R (ggplot2)	Python (plotnine)
Data	`ggplot(df, ...`	`ggplot(df, ...`
Aesthetics	`aes(x=displ, y=hwy, color=class)`	`aes(x="displ", y="hwy", color="class")`
Geometry	`+ geom_point(size=2, alpha=0.6)`	`+ geom_point(size=2, alpha=0.6)`
Smooth	`+ geom_smooth(method='lm')`	`+ geom_smooth(method='lm')`
Scale	`+ scale_color_brewer(palette='Set2')`	`+ scale_color_brewer(type='qual',palette='Set2')`
Facet	`+ facet_wrap(~cyl, ncol=2)`	`+ facet_wrap('~cyl', ncol=2)`
Labels	`+ labs(title='...', x='...')`	`+ labs(title='...', x='...')`
Theme	`+ theme_minimal()`	`+ theme_minimal()`
Save	`ggsave('p.pdf', width=7)`	`p.save('p.pdf', width=7, height=5)`

plotnine: Complete Example

```
from plotnine import *
import pandas as pd

apps = pd.read_csv(
    "googleplaystore.csv")

(ggplot(apps.dropna(
        subset=["Rating"]),
    aes(x="Reviews", y="Rating",
        color="Type"))

 + geom_point(alpha=0.3, size=0.8)

 + geom_smooth(method="lm",
     se=True, color="grey")

 + scale_x_log10()

 + scale_color_manual(
     values={"Free": "#1565C0",
             "Paid": "#E53935"})

 + facet_wrap("~Content_Rating",
     ncol=2)

 + theme_minimal()
```



Every ggplot2 feature has a plotnine equivalent: `aes()`, `geom_*()`, `scale_*()`, `facet_*()`, `theme_*()`, `labs()`. The only differences are strings for column names and + at line start.

plotnine vs ggplot2: The Three Differences

Feature	R (ggplot2)	Python (plotnine)
Column names	Bare symbols: `aes(x = displ)`	Strings: `aes(x="displ`
Line continuation	+ at end of line	+ at start, inside ()
Save	`ggsave("p.pdf")`	`p.save("p.pdf")`

Data Insight

Everything else — geom names, scale functions, facet syntax, theme elements — is character-for-character identical. If you can write ggplot2, you can write plotnine in under 5 minutes.

Rosetta Stone

R ↔ Python Translation Table

R ↔ Python Rosetta Stone: ggplot2 / seaborn / matplotlib

Task	R (ggplot2)	Python (seaborn)	Python (matplotlib)
Scatter + color	`geom_point(aes(color=g))`	`sns.scatterplot(hue='g')`	`ax.scatter(c=colors)`
Histogram	`geom_histogram(bins=30)`	`sns.histplot(bins=30)`	`ax.hist(bins=30)`
Boxplot	`geom_boxplot(fill=g)`	`sns.boxplot(hue='g')`	`ax.boxplot(patch_artist)`
Line + CI	`geom_smooth(method='lm')`	`sns.regplot(ci=95)`	`ax.plot() + fill_between`
Facet wrap	`facet_wrap(~g, ncol=3)`	`col='g', col_wrap=3`	`plt.subplots() loop`
Facet grid	`facet_grid(r~c)`	`row='r', col='c'`	`plt.subplots(nr,nc)`
Theme	`theme_minimal()`	`sns.set_theme('whitegrid')`	`rcParams + manual`

Same Chart, Three Ways

R (ggplot2)

```r
ggplot(df, aes(
    x = displ, y = hwy
        ,
    color = class)) +
  geom_point(alpha
      =0.6) +
  geom_smooth(
    method = "lm") +
  scale_color_brewer(
    palette = "Set2")
        +
  facet_wrap(~cyl) +
  theme_minimal()
```

Python (plotnine)

```python
(ggplot(df, aes(
    x="displ", y="hwy"
        ,
    color="class"))
 + geom_point(alpha
     =0.6)
 + geom_smooth(
     method="lm")
 + scale_color_brewer(
     type="qual",
     palette="Set2")
 + facet_wrap("~cyl")
 + theme_minimal())
```

Python (seaborn)

```python
g = sns.lmplot(
    data=df,
    x="displ",
    y="hwy",
    hue="class",
    col="cyl",
    col_wrap=2,
    palette="Set2",
    height=3,
    aspect=1.2,
    ci=95,
    scatter_kws={
      "alpha": 0.6})
```

Synthesis

Library Selection Summary

Use Case	Recommended Library
Want ggplot2 syntax in Python	`plotnine`
Statistical charts with auto-aggregation	`seaborn`
Faceted statistical charts	`seaborn` (figure-level) or `plotnine`
Custom multi-panel dashboard	`matplotlib` + `GridSpec`
Embed seaborn in custom layout	`seaborn` axes-level + `ax=`
Heatmaps, pairplots, clusterplots	`seaborn`
Maximum customisation	`matplotlib`

Key Takeaways

1. **matplotlib**: low-level, full control, verbose. **seaborn**: statistical, auto-aggregation, faceting. **plotnine**: ggplot2 clone, grammar syntax.
2. seaborn has two API levels: **figure-level** (creates figure, has faceting) and **axes-level** (draws on existing ax).
3. seaborn's **hue/style/size** map to ggplot2's **color/shape/size** aesthetics.
4. plotnine syntax is **identical** to ggplot2 except: strings for column names, + at line start, `.save()`.
5. The **Rosetta Stone** table translates any ggplot2 chart to seaborn or matplotlib with known equivalences.

Essential Readings

- seaborn tutorial: `https://seaborn.pydata.org/tutorial.html`
- plotnine documentation: `https://plotnine.org`
- VanderPlas, J. (2016). *Python Data Science Handbook*, Chapter 4 (Visualization with Matplotlib).
- Waskom, M. (2021). "seaborn: statistical data visualization." *JOSS*, 6(60), 3021.

Next Module: labs(), Annotations & Storytelling Layers (W02-M08)